

Rilevamento e mitigazione dinamica di attacchi DoS in reti SDN

Network and Cloud Infrastructures - 2026

Studente: Michele Contino

Matricola: DE9000273

1. Overview e obiettivi

Il progetto analizza l'utilizzo delle caratteristiche programmabili di una rete *Software Defined Networking* per rilevare e mitigare automaticamente un attacco *Denial of Service*.

Lo scenario considera un host malevolo, h1, che genera traffico UDP ad elevata bitrate verso un server h3, condividendo parte dell'infrastruttura con il client legittimo h2. L'attacco genera congestione sul collegamento comune e provoca un degrado delle prestazioni di h2 in termini di throughput, latenza e packet loss.

Lo sviluppo è stato organizzato in versioni progressive: **Base**, **Monitoring**, **Detection**, **Mitigation** e **Dynamic Mitigation**. L'ultima versione introduce inoltre lo sblocco automatico dopo la fine dell'anomalia e una detection basata su più campioni consecutivi, in modo da non reagire immediatamente a un singolo picco temporaneo.

2. Ambiente e topologia

Gli esperimenti sono stati eseguiti su Ubuntu 22.04 utilizzando **Mininet** per l'emulazione della rete, **Open vSwitch** come switch software, **OpenFlow 1.3** come protocollo tra data plane e control plane e **Ryu** per l'implementazione dei controller. **iperf** è stato utilizzato per generare traffico TCP e UDP, mentre **ping** è stato utilizzato per misurare RTT e packet loss.

La topologia comprende tre host e quattro switch:

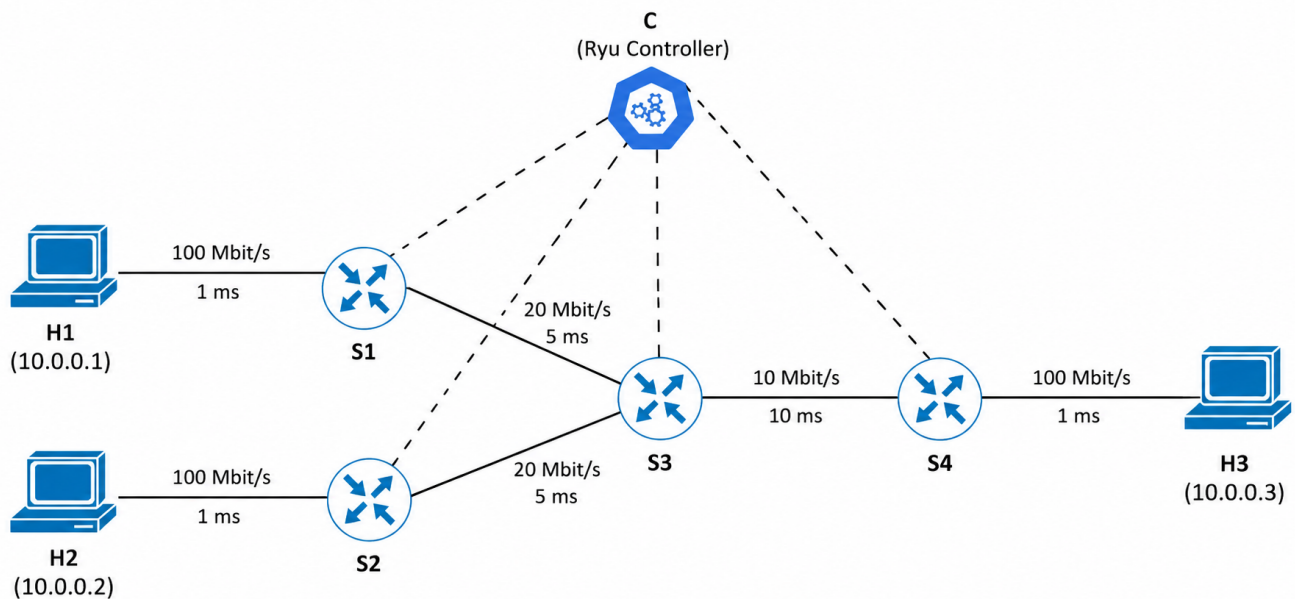


Figura 1: Topologia SDN utilizzata negli esperimenti.

Gli indirizzi degli host sono: h1 10.0.0.1, h2 10.0.0.2, h3 10.0.0.3.

I link host-switch hanno capacità 100 Mbit/s e delay 1 ms. I collegamenti s1-s3 e s2-s3 sono configurati a 20 Mbit/s e 5 ms, mentre s3-s4 è limitato a 10 Mbit/s con delay 10 ms e rappresenta il collo di bottiglia condiviso. Durante l'attacco, sulla porta di s1 collegata ad h1 sono stati rilevati valori stabili di circa 16.18 Mbit/s. Il valore risulta leggermente superiore ai 15 Mbit/s impostati tramite iperf poiché le statistiche della porta comprendono anche l'overhead introdotto dai protocolli di rete. Sul collegamento verso s4 il traffico si stabilizza invece intorno a 9.95 Mbit/s, in accordo con il limite di 10 Mbit/s imposto sul link s3-s4. Queste misure confermano la corretta acquisizione delle statistiche e la presenza del bottleneck previsto dalla topologia.

3. Sviluppo dei controller

3.1 Controller base

`controller_base.py` implementa un learning switch OpenFlow 1.3. Alla connessione di ciascuno switch viene installata una regola *table-miss* che invia al controller i pacchetti non ancora associati a una flow entry. Alla ricezione di un evento `PacketIn`, il controller apprende l'associazione tra MAC sorgente e porta di ingresso tramite la struttura `mac_to_port`. Se il MAC di destinazione è noto viene selezionata la porta corretta; in caso contrario viene eseguito il flooding. Quando la destinazione è conosciuta, una regola OpenFlow con priorità 1 viene installata sullo switch, consentendo ai pacchetti successivi di essere gestiti direttamente dal data plane.

Questa versione non possiede meccanismi di controllo del traffico e costituisce lo scenario di riferimento senza protezione.

3.2 Controller con monitoring

`controller_monitoring.py` estende il learning switch introducendo il monitoraggio periodico delle porte. Il controller mantiene l'elenco dei datapath connessi e avvia un thread tramite `hub.spawn` che, ogni 3 secondi, invia a ogni switch un messaggio `OFPPortStatsRequest`.

Le risposte `EventOFPPortStatsReply` forniscono, tra le altre informazioni, il contatore cumulativo `rx_bytes`. Per ricavare il traffico nell'intervallo corrente viene memorizzato il campione precedente e calcolato:

$$R = \frac{(B_{new} - B_{old}) \cdot 8}{(t_{new} - t_{old}) \cdot 10^6} \quad [Mbit/s]$$

Il controller può quindi mostrare valori quali:

```
s1 port 1 rx 14.5 Mbit/s
s2 port 1 rx 2.1 Mbit/s
```

Questa versione è solamente osservativa: non genera allarmi e non modifica le flow table. Le misure ottenute costituiscono la base per la successiva fase di detection.

3.3 Controller con detection

`controller_detection.py` confronta il throughput misurato sulle porte edge con una soglia iniziale impostata a:

$$T = 12 \text{ Mbit/s}$$

La soglia viene applicata alle porte edge direttamente collegate agli host e non al collegamento `s3-s4`, la cui capacità è limitata a 10 Mbit/s. Il valore di 12 Mbit/s è stato scelto sulla base dello scenario sperimentale e successivamente validato attraverso le misure: durante l'attacco la porta di `s1` collegata ad `h1` riceve circa 16.18 Mbit/s, mentre in condizioni normali il traffico osservato rimane inferiore alla soglia. In questo modo è possibile distinguere il comportamento anomalo dell'attaccante senza generare allarmi durante il normale funzionamento della rete.

Sono controllate in particolare le porte direttamente associate agli host, ad esempio `(s1,1)` per `h1` e `(s2,1)` per `h2`. In questo modo l'anomalia viene collegata alla sorgente del traffico e non a una porta interna che trasporta flussi aggregati.

Quando il valore misurato supera la soglia il controller genera un messaggio di allarme:

```
ALARM possible DoS from h1
```

Quando il valore misurato supera la soglia, il controller genera un messaggio di allarme e aggiorna lo stato globale `alarm`, che passa a `True` quando almeno una delle porte monitorate presenta un throughput anomalo. In questa fase non viene ancora applicata alcuna remediation: il traffico dell'attaccante continua quindi a raggiungere `h3`. La separazione tra detection e mitigation permette di verificare indipendentemente il corretto riconoscimento dell'anomalia prima di modificare il comportamento del data plane.

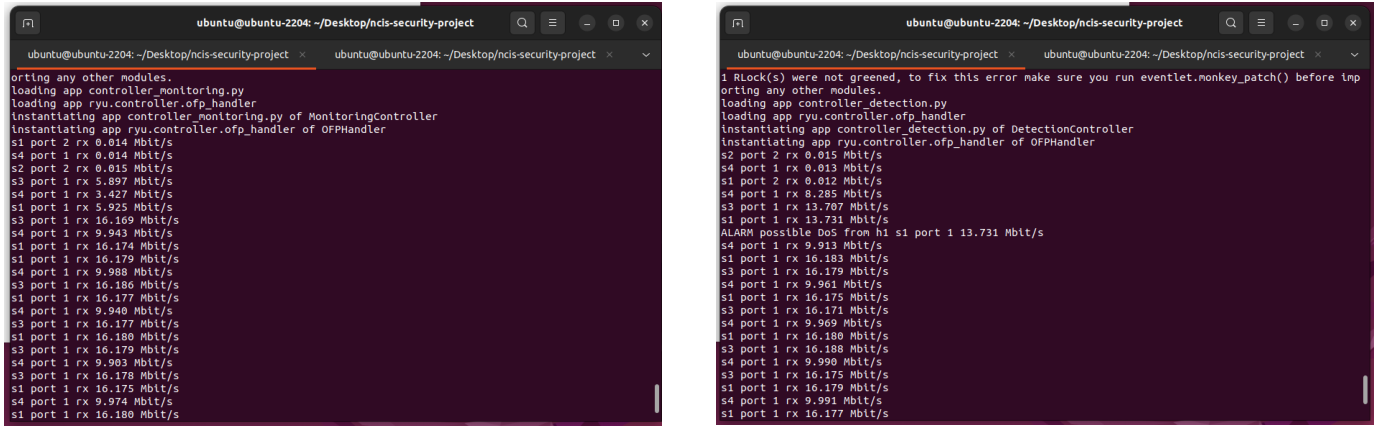


Figura 2: A sinistra, statistiche delle porte raccolte dal controller di monitoring; a destra, generazione dell'allarme in seguito al superamento della soglia.

3.4 Controller con mitigation

`controller_mitigation.py` completa il meccanismo base aggiungendo una remediation automatica. Quando viene rilevato traffico anomalo, il controller identifica lo switch e la porta associati alla sorgente e installa una regola di drop tramite `OFPFLOWMod`.

Nel caso di `h1` viene utilizzato un match sulla porta di ingresso:

```
match = parser.OFPMatch(in_port=in_port)
```

La nuova flow entry ha priorità 100 e non contiene azioni di forwarding. Di conseguenza i pacchetti corrispondenti vengono scartati. La priorità è maggiore rispetto alle normali regole del learning switch:

$$100_{drop} > 1_{forwarding}$$

La struttura `blocked_ports` evita di reinstallare la stessa regola a ogni intervallo di monitoring.

Il blocco viene applicato direttamente su `s1`:

$$h1 \rightarrow s1 \rightarrow DROP$$

Il traffico dell'attaccante non raggiunge quindi più il collo di bottiglia `s3-s4`, lasciando nuovamente capacità disponibile per `h2`.

È possibile che `rx_bytes` sulla porta di `h1` continui ad aumentare anche durante il blocco: il contatore registra infatti i frame ricevuti dalla porta prima dell'applicazione della flow table. Il corretto funzionamento viene quindi verificato osservando la regola installata su `s1` e il recupero delle prestazioni del client legittimo.

3.5 Controller con mitigazione dinamica

`controller_dynamic_mitigation.py` introduce due funzionalità aggiuntive.

La prima evita che un singolo picco temporaneo provochi immediatamente il blocco. Il controller richiede tre misure consecutive sopra soglia:

```
ATTACK_SAMPLES = 3
```

Il contatore `high_samples` viene incrementato per ogni campione anomalo e azzerato se il traffico torna normale prima di raggiungere il valore richiesto:

$$alto \rightarrow alto \rightarrow alto \rightarrow BLOCK$$

In questo modo la detection considera la persistenza dell'anomalia e riduce la sensibilità a picchi isolati.

La seconda funzionalità realizza lo sblocco automatico. Dopo la mitigation, se `h1` interrompe o riduce l'attacco e il traffico resta sotto soglia per tre campioni consecutivi,

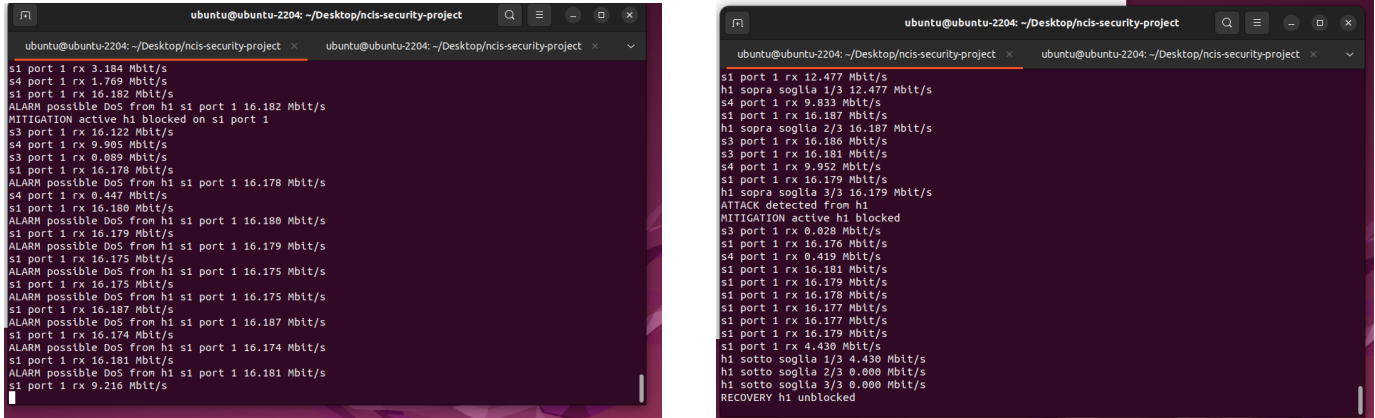
```
RECOVERY_SAMPLES = 3
```

il controller elimina la regola di drop con `OFFPFC_DELETE_STRICT`, rimuove la porta da `blocked_ports` e azzeri i contatori.

Il comportamento finale può essere sintetizzato come:

NORMAL → 3 HIGH → DETECTION → BLOCK → 3 LOW → UNBLOCK

Questa versione rende quindi la risposta automatica sia nella fase di mitigazione sia nella fase di recovery.



The image contains two terminal windows side-by-side. The left window shows a sequence of log messages: 's1 port 1 rx 3.184 Mbit/s', 's4 port 1 rx 1.769 Mbit/s', 's1 port 1 rx 16.182 Mbit/s', 'ALARM possible DoS from h1 s1 port 1 16.182 Mbit/s', 'MITIGATION active h1 blocked on s1 port 1', and then several 's3 port 1 rx' and 's4 port 1 rx' entries. The right window shows a similar log sequence but includes threshold checks: 'h1 sopra soglia 1/3 12.477 Mbit/s', 's4 port 1 rx 9.833 Mbit/s', 's1 port 1 rx 16.187 Mbit/s', 'h1 sopra soglia 2/3 16.187 Mbit/s', 's3 port 1 rx 16.186 Mbit/s', 's4 port 1 rx 9.952 Mbit/s', 's1 port 1 rx 16.179 Mbit/s', 'h1 sopra soglia 3/3 16.179 Mbit/s', 'ATTACK detected from h1', 'MITIGATION active h1 blocked', and finally 'RECOVERY h1 unblocked'.

Figura 3: A sinistra, attivazione della mitigation e blocco del traffico proveniente da h1; a destra, comportamento del controller dinamico durante la detection persistente e il successivo recovery automatico.

4. Risultati sperimentali

La validazione è stata effettuata mantenendo invariati la topologia e lo scenario di traffico e utilizzando progressivamente le diverse versioni del controller. Il traffico legittimo è generato da h2 verso h3 tramite TCP, mentre durante l'attacco h1 genera verso lo stesso server un flusso UDP a 15 Mbit/s.

In condizioni normali, utilizzando `controller_base.py`, il traffico h2–h3 ha raggiunto un throughput di **9.49 Mbit/s**, con RTT medio pari a **36.60 ms** e packet loss nullo. Il risultato è coerente con la capacità di 10 Mbit/s del collegamento s3–s4, che rappresenta il collo di bottiglia del percorso.

Introducendo il traffico UDP di h1 senza alcun meccanismo di protezione, il throughput TCP del client legittimo è sceso a **0.0352 Mbit/s**, mentre l'RTT medio è salito a **144.74 ms** e il packet loss ha raggiunto il **70%**. L'attacco provoca quindi una forte degradazione del servizio offerto ad h2.

Scenario	Throughput h2	RTT medio	Packet loss
Normale	9.49 Mbit/s	36.60 ms	0%
DoS senza protezione	0.0352 Mbit/s	144.74 ms	70%
DoS + mitigation	9.49 Mbit/s	35.98 ms	0%

Tabella 1: Prestazioni del traffico legittimo h2–h3 nei diversi scenari.

Il corretto funzionamento del monitoring è stato verificato osservando i valori calcolati dal controller a partire dalle statistiche OpenFlow. Durante l'attacco, sulla porta di s1 collegata ad h1 sono stati rilevati valori stabili di circa **16.18 Mbit/s**, mentre sul collegamento verso s4 il traffico si è stabilizzato intorno a **9.95 Mbit/s**, in accordo con il limite di 10 Mbit/s imposto sul link s3–s4. Queste misure confermano sia la corretta acquisizione delle statistiche sia la presenza del collo di bottiglia previsto dalla topologia.

I valori osservati hanno inoltre consentito di validare la soglia di **12 Mbit/s** utilizzata dal controller di detection: il traffico normale si mantiene intorno a 9.5–10 Mbit/s, mentre il traffico generato da h1 durante l'attacco supera stabilmente la soglia. Con `controller_detection.py` il superamento di tale valore provoca la generazione dell'allarme, senza però modificare il forwarding.

Con `controller_mitigation.py`, al rilevamento dell'anomalia viene installata su s1 una flow entry con priorità 100 e azione di drop sulla porta di ingresso di h1. La presenza della regola è stata verificata

direttamente nella flow table dello switch:

```
priority=100,in_port="s1-eth1" actions=drop
```

L'efficacia della contromisura emerge dal confronto riportato in Tabella 1: nonostante h1 continui a generare il traffico malevolo, il throughput di h2 torna a **9.49 Mbit/s**, l'RTT medio scende a **35.98 ms** e il packet loss torna allo **0%**. Le prestazioni risultano quindi sostanzialmente equivalenti alla baseline.

Infine, `controller_dynamic_mitigation.py` è stato verificato controllando le due funzionalità aggiuntive introdotte. Un picco isolato non sufficiente a produrre tre campioni consecutivi sopra soglia non provoca il blocco, mentre un traffico anomalo persistente porta alla progressione 1/3, 2/3, 3/3 e alla successiva attivazione della mitigation. Dopo la cessazione dell'attacco, tre campioni consecutivi sotto soglia determinano la rimozione automatica della regola di drop e il ripristino della normale connettività di h1.

5. Conclusioni

Il progetto mostra come un controller SDN possa utilizzare informazioni raccolte dal data plane per implementare una politica automatica di sicurezza. Lo sviluppo incrementale ha permesso di separare chiaramente le fasi di forwarding, monitoring, detection e mitigation.

Le statistiche OpenFlow consentono al controller di stimare il throughput delle porte; il confronto con una soglia permette di individuare traffico anomalo, mentre l'installazione dinamica di una flow entry di drop impedisce all'attaccante di raggiungere il collo di bottiglia condiviso.

La versione finale aggiunge una detection basata sulla persistenza dell'anomalia e un meccanismo automatico di recovery. Un singolo picco non determina quindi immediatamente un blocco e la regola di mitigazione viene rimossa quando il traffico torna stabilmente a valori normali.

Come possibile sviluppo futuro, il blocco potrebbe essere reso più selettivo utilizzando campi IP e di trasporto, limitando esclusivamente il flusso malevolo invece dell'intera porta. Un'ulteriore evoluzione potrebbe utilizzare una soglia dinamica ricavata dal comportamento storico della rete.

Esecuzione. Il controller viene avviato con `ryu-manager nome_controller.py`, mentre la topologia viene eseguita con `sudo python3 topology.py`. I comandi completi per la generazione del traffico, gli esperimenti iperf/ping e la pulizia dell'ambiente sono riportati nel file di istruzioni incluso nel repository.